## Project NUDTEE – Team Dek70

**Member** 1.6931503050 Ninmanee Nualsee
2.6931503065 Rawisara Palang
3.6931503119 Suratsawadee Areerob
4.6931503074 Sirawit Kansuwan
5.6931503107 Thanyarat Ainpasat

## Problem

General activity scheduling often suffers from unclear timing and participant requirements. Critical details like location and schedules get buried in group chats, leading to confusion and forgotten appointments.

## Who has this problem?

Groups wanting to organize activities and the organizers who need to track participant numbers.

## Why it matters / what it costs them

Time wasted scrolling through chat histories, missed opportunities due to delayed coordination, and the hassle of following up with people who forget the schedule.

## Gate 1 — Real users

1. Ms. Rawipriya Udomplaa (student) tel-0887741637 2 November 2569
2. Ms. Jeeranan Yepiang (student) tel-0946208353 2 November 2569
3. Mr. Apiwat Chaiswat (student) tel-0955687886 2 November 2569

## Gate 2 — MVP scope

**FR-1 Create / edit a session** A user creates a session with a title, location (or an online meeting link), date/time, minimum/maximum players, a registration deadline, and a category.

**FR-2 Join / leave a session** A user joins or leaves a session; the system counts joined players and computes remaining spots.

**FR-3 Real-time session status** The session details and participant count update instantly when someone joins or leaves, with no refresh.

**FR-4 Display location on map** A user views the session's location on an interactive map.

**FR-5 In-room chat** A user sends and receives messages in real time within the specific session's chat room.

**FR-6 Auto-close session** The system automatically blocks new users from joining once the registration deadline is reached.

**FR-7 Add to Google Calendar** A user adds a joined session to their Google Calendar through a pre-filled event.

## Which FRs contain real logic or decisions

**FR-2 (Join / leave a session):** remaining spots = max_players – current_joined; a join must be rejected when the session is full. Includes race condition handling (e.g., if two users tap join at the exact same time with only one spot left, only one may succeed), done with a database transaction.

**FR-3 (Real-time session status):** State must be synchronized in real time across many users within 2 seconds through database real-time listeners, and clients must resynchronize after a connection loss (handled automatically by the Firebase listeners).

**FR-6 (Auto-close session):** The system must compare the current time with the session's registration deadline (end of the deadline day) and start time; it must contain logic to reject any join attempts once registration has closed.

## Out of scope this term

**Waitlist Management:** Managing an automated queue or waitlist for users attempting to join a session that is already full.

**Push Notifications & Native Reminders:** Sending in-app push notifications, SMS, or email alerts (schedule reminders are handled entirely via the external Google Calendar).

**User Ratings and Reviews:** Rating or reviewing session organizers, participants, or venue locations.

**Venue Booking and Payment Integration:** Direct booking mechanisms with commercial venues or cafes, as well as in-app payment processing or bill splitting**.**

**Social & Direct Messaging System:** Friend list management, user profile following, social feed features, and private 1-on-1 direct messaging outside of session-specific chat rooms.

**Editing a Session After Creation:** The organizer closes the room and creates a new one instead.

**Map Overview of All Sessions:** Pins and summary cards for all sessions on one map (a location is viewed per session).

**Calendar API Integration:** Handling Google OAuth access tokens or Calendar API calls inside NUDTEE (the export uses a pre-filled Google Calendar event link).

## Data that must persist

**User profile:** Google account ID and display name (from Google sign-in), plus the optional profile fields the user enters (username, short bio, profile picture). The user's own full profile is kept in the browser's local storage; display name and a small picture thumbnail are copied into the participation record.

**Session details:** Title, short and room descriptions, category, online/offline type, location name, address and coordinates (or meeting link), date, start/end time, minimum/maximum players, registration deadline, and cover image.

**Participation records:** The participant list stored with each session (which user has joined which session), used to calculate remaining spots and reject joins when the session is full.

**Chat data:** Messages sent within each session's in-room chat, including sender ID, name, and timestamps.

## Success metrics

| ID | Metric | Target |
|---|---|---|
| SM-1 | Time for an organizer to create a session | 60 seconds or less |
| SM-2 | Taps for a player to join an open session from the list | 2 taps or fewer |
| SM-3 | Time for another device to show a join/leave change | 2 seconds or less |
| SM-4 | Taps to add a joined session to Google Calendar (excluding Google sign-in) | 2 taps or fewer |
| SM-5 | Rejection rate for join attempts (through the application) after registration has closed | 100% |
| SM-6 | Delivery time for in-room chat messages across devices | 2 seconds or less |

## Gate 3 — Access and dependencies

| Dependency | How we confirm it is really accessible |
|---|---|
| Auth provider (Google sign-in through Firebase Authentication) | Create a Firebase project, enable the Google sign-in provider and add the app's domain as an authorized domain; check that it is free on the Spark plan and what the limits are. |
| Realtime database (Cloud Firestore, Firebase free Spark plan) | Check the free quotas against the expected usage; set up a test project, send a realtime update, and confirm latency is about 2 seconds or better. |
| Map display (embedded Google Maps view) and place search (OpenStreetMap-based Nominatim and Photon services) | Confirm the embedded map loads without a paid API key, and that the search endpoints are open, free of charge, and used within their usage policies (light, rate-limited requests). |
| Google Calendar (event-template link) | Confirm that opening the link pre-fills the event title, start/end time, location, and description, and that no API activation, OAuth token, or cost is needed. |
| Hosting (Vercel / Netlify) | Deploy a test environment from a free account and confirm no paywall or timeout issues block the real-time connections and external requests. |

## Role

| Role | Responsibility | Owner |
|---|---|---|
| Product/project lead | Scope,planning | -Sirawit Kansuwan |
| Documetation | SRS,diagrams | -Sirawit Kansuwan<br>-Suratsawadee Areerob<br>-Thanyarat Ainpasat |
| Design/Frond/UI | Screens and clickable prototype,design doc | -Ninmanee Nualsee<br>-Rawisara Palang |
| Backend/data | Firebase (Authentication, Firestore), join logic, realtime | -Thanyarat Ainpasat<br>-Ninmanee Nualsee |
| QA/AI log | Test results,AI usage log | -Suratsawadee Areerob<br>-Rawisara Palang |